Отчёт по лабораторной работе №3

Дисциплина: Компьютерный практикум по статистическому анализу данных

Выполнил: Танрибергенов Эльдар (НПИбд-01-22)

Содержание

1 Цель работы **6**

2 Выполнение лабораторной работы **7**

2.1 Циклы while и for 7
2.2 Условные выражения 10
2.3 Функции 12
2.4 Сторонние библиотеки (пакеты) в Julia 16
2.5 Задания для самостоятельного выполнения 19

3 Выводы **29**

Список иллюстраций

2.1 Вывод чисел от 1 до 10 (цикл while) 7
2.2 Обращение к элементам массива (цикл while) 8
2.3 Вывод чисел от 1 до 10 (цикл for) 8
2.4 Обращение к элементам массива (цикл for) 9
2.5 Использование цикла для создания массива 9
2.6 Другая реализация предыдущего примера 10
2.7 Пример исп. условных выражений 11
2.8 Пример исп. тернарного оператора 11
2.9 Пример функции 12
2.10 Функция возведения в квадрат 12
2.11 Примеры вызовов функций 12
2.12 Пример однострочных функций 13
2.13 Пример анонимных функций 13
2.14 Применение функций с/без сохранения изменений 13
2.15 Передача функций в функцию 14
2.16 Передача анонимно заданной ф-ции 14
2.17 Функция broadcast 15
2.18 Пример функции возведения в кв. 15
2.19 Пример работы функции broadcast 15
2.20 Варианты записи сложных выражений 16
2.21 Добавление стороннего пакета 17
2.22 Создание матрицы цветов 18
2.23 Задание №1.1: цикл for 19
2.24 Задание №1.1: цикл while 19
2.25 Задание №1.2: цикл for 20
2.26 Задание №1.2: цикл while 20
2.27 Задание №1.3: цикл for 21
2.28 Задание №1.3: цикл while 21
2.29 Задание №2: результат 1 21
2.30 Задание №2: результат 2 22
2.31 Задание №3 22
2.32 Задание №4 22
2.33 Задание №5.1 23
2.34 Задание №5.2 23
2.35 Задание №6 23
2.36 Задание №7.1 24
2.37 Задание №7.2 25

2.38 Задание №7.3 25
2.39 Задание №8 26
2.40 Задание №9 26
2.41 Задание №10 27
2.42 Задание №11 28

Список таблиц

1 Цель работы

Основная цель работы — освоить применение циклов функций и сторонних для Julia пакетов для решения задач линейной алгебры и работы с матрицами.

2 Выполнение лабораторной работы

2.1 Циклы while и for

Для различных операций, связанных с перебором индексируемых элементов структур данных, традиционно используются циклы while и for. Синтаксис while

```
while <условие>
    <тело цикла>
end
```

Например, while можно использовать для формирования элементов массива:

- Пока n<10 прибавить к n единицу и распечатать значение:

```
[1]: n = 0
     while n < 10
     n += 1
     println(n)
     end

     1
     2
     3
     4
     5
     6
     7
     8
     9
     10
```

Рис. 2.1: Вывод чисел от 1 до 10 (цикл while)

Другой пример демонстрирует использование while при работе со строковыми элементами массива, подставляя имя из массива в заданную строку приветствия и выводя получившуюся конструкцию на экран:

```
[2]: myfriends = ["Ted", "Robyn", "Barney", "Lily", "Marshall"]
     i = 1
     while i <= length(myfriends)
     friend = myfriends[i]
     println("Hi $friend, it's great to see you!")
     i += 1
     end

     Hi Ted, it's great to see you!
     Hi Robyn, it's great to see you!
     Hi Barney, it's great to see you!
     Hi Lily, it's great to see you!
     Hi Marshall, it's great to see you!
```

Рис. 2.2: Обращение к элементам массива (цикл while)

Такие же результаты можно получить при использовании цикла for. Синтаксис for

```
for <переменная> in <диапазон>
    <тело цикла>
end
```

Рассмотренные выше примеры, но с использованием цикла for:

```
[3]: for n in 1:2:10
     println(n)
     end

     1
     3
     5
     7
     9
```

Рис. 2.3: Вывод чисел от 1 до 10 (цикл for)

```
[4]: myfriends = ["Ted", "Robyn", "Barney", "Lily", "Marshall"]
for friend in myfriends
println("Hi $friend, it's great to see you!")
end
```

```
Hi Ted, it's great to see you!
Hi Robyn, it's great to see you!
Hi Barney, it's great to see you!
Hi Lily, it's great to see you!
Hi Marshall, it's great to see you!
```

Рис. 2.4: Обращение к элементам массива (цикл for)

Пример использования цикла for для создания двумерного массива, в котором значение каждой записи является суммой индексов строки и столбца:

- Инициализация массива m x n из нулей. Формирование массива, в котором значение каждой записи является суммой индексов строки и столбца:

```
[7]: m, n = 5, 5
A = fill(0, (m, n))
for i in 1:m
for j in 1:n
A[i, j] = i + j
end
end
A
```

```
[7]: 5×5 Matrix{Int64}:
 2  3  4  5   6
 3  4  5  6   7
 4  5  6  7   8
 5  6  7  8   9
 6  7  8  9  10
```

Рис. 2.5: Использование цикла для создания массива

Другая реализация этого же примера:

```
[9]: B = fill(0, (m, n))
     for i in 1:m, j in 1:n
     B[i, j] = i + j
     end
     B
     C = [i + j for i in 1:m, j in 1:n]
     C
```

```
[9]: 5×5 Matrix{Int64}:
      2  3  4  5   6
      3  4  5  6   7
      4  5  6  7   8
      5  6  7  8   9
      6  7  8  9  10
```

Рис. 2.6: Другая реализация предыдущего примера

2.2 Условные выражения

Довольно часто при решении задач требуется проверить выполнение тех или иных условий. Для этого используют условные выражения. Синтаксис условных выражений с ключевым словом:

```
if <условие 1>
    <действие 1>
elseif <условие 2>
    <действие 2>
else
    <действие 3>
end
```

Например, пусть для заданного числа требуется вывести слово «Fizz», если делится на 3, «Buzz», если делится на 5, и «FizzBuzz», если делится на 3 и 5. Используем && для реализации операции “AND”. Операция % вычисляет остаток от деления:

```
[12]: println("Введите число: ")
      N = parse(Int, readline())
      if (N % 3 == 0) && (N % 5 == 0)
      println("FizzBuzz")
      elseif N % 3 == 0
      println("Fizz")
      elseif N % 5 == 0
      println("Buzz")
      else
      println(N)
      end

      Введите число:
      stdin>  15
      FizzBuzz
```

Рис. 2.7: Пример исп. условных выражений

Синтаксис условных выражений с тернарными операторами:

`a ? b : c`

(если выполнено a, то выполнить b, если нет, то c).

Такая запись эквивалентна записи условного выражения с ключевым словом:

```
if a
    b
else
    c
end
```

Пример использования тернарного оператора:

```
[13]: x = 5
      y = 10
      (x > y) ? x : y

[13]: 10
```

Рис. 2.8: Пример исп. тернарного оператора

2.3 Функции

Julia дает нам несколько разных способов написать функцию. Первый требует ключевых слов function и end:

```
[14]: function sayhi(name)
      println("Hi $name, it's great to see you!")
      end

[14]: sayhi (generic function with 1 method)
```

Рис. 2.9: Пример функции

- функция возведения в квадрат:

```
[15]: function f(x)
      x^2
      end

[15]: f (generic function with 1 method)
```

Рис. 2.10: Функция возведения в квадрат

Вызов функции осуществляется по её имени с указанием аргументов, например:

```
[16]: sayhi("C-3PO")
      f(42)

      Hi C-3PO, it's great to see you!
[16]: 1764
```

Рис. 2.11: Примеры вызовов функций

В качестве альтернативы, можно объявить любую из выше определённых функций в одной строке:

```
[17]: sayhi2(name) = println("Hi $name, it's great to see you!")
      f2(x) = x^2
```

Рис. 2.12: Пример однострочных функций

Наконец, можно объявить выше определённые функции как «анонимные»:

```
[18]: sayhi3 = name -> println("Hi $name, it's great to see you!")
      f3 = x -> x^2

[18]: #8 (generic function with 1 method)
```

Рис. 2.13: Пример анонимных функций

По соглашению в Julia функции, сопровождаемые восклицательным знаком, изменяют свое содержимое, а функции без восклицательного знака не делают этого.

Например, сравнил результат применения sort и sort!:

```
[21]: v = [3, 5, 2]
      sort(v)
      println(v)
      sort!(v)
      println(v)

      [3, 5, 2]
      [2, 3, 5]
```

Рис. 2.14: Применение функций с/без сохранения изменений

Функция sort(v) возвращает отсортированный массив, который содержит те же элементы, что и массив v, но исходный массив v остаётся без изменений. Если же использовать sort!(v), то отсортировано будет содержимое исходного массива v. В программировании под функцией высшего порядка понимается функция, принимающая в качестве аргументов другие функции или возвращающая другую функцию в качестве результата. Основная идея состоит в том, что функции имеют тот же статус, что и другие объекты данных. В Julia функция map является функцией высшего порядка, которая принимает функцию в качестве одного из

своих входных аргументов и применяет эту функцию к каждому элементу структуры данных, которая ей передаётся также в качестве аргумента. Например, в случае выполнения выражения: `map(f, [1, 2, 3])` на выходе получим массив, в котором функция f была применена ко всем элементам массива [1, 2, 3]: `[f(1), f(2), f(3)]` Если $f(x) = x2$, то получим следующий результат:

```
[23]: f(x) = x^2
      map(f, [1, 2, 3])

[23]: 3-element Vector{Int64}:
       1
       4
       9
```

Рис. 2.15: Передача функций в функцию

То есть в квадрат возведены все элементы массива [1, 2, 3], но не сам массив (вектор). В map можно передать и анонимно заданную, а не именованную функцию:

```
[25]: x -> x^3
      map(x -> x^3, [1, 2, 3])

[25]: 3-element Vector{Int64}:
        1
        8
       27
```

Рис. 2.16: Передача анонимно заданной ф-ции

Функция broadcast — ещё одна функция высшего порядка в Julia, представляющая собой обобщение функции map. Функция broadcast() будет пытаться привести все объекты к общему измерению, map() будет напрямую применять данную функцию поэлементно. Синтаксис для вызова broadcast такой же, как и для вызовамap, например применение функции f к элементам массива [1, 2, 3]:

```
[26]: f(x) = x^2
      broadcast(f, [1, 2, 3])
      f.([1, 2, 3])

[26]: 3-element Vector{Int64}:
       1
       4
       9
```

Рис. 2.17: Функция broadcast

Пример:

- Задаём матрицу A. Вызываем функцию f возведения в квадрат.

```
[27]: A = [i + 3*j for j in 0:2, i in 1:3]

[27]: 3×3 Matrix{Int64}:
       1  2  3
       4  5  6
       7  8  9

[28]: f(A)

[28]: 3×3 Matrix{Int64}:
        30   36   42
        66   81   96
       102  126  150
```

Рис. 2.18: Пример функции возведения в кв.

С другой стороны B = f.(A) Результат (содержит квадраты всех элементов A):

```
[29]: B = f.(A)

[29]: 3×3 Matrix{Int64}:
        1   4   9
       16  25  36
       49  64  81
```

Рис. 2.19: Пример работы функции broadcast

Точечный синтаксис для broadcast() позволяет записать относительно сложные составные поэлементные выражения в форме, близкой к математической записи:

```
[30]: A .+ 2 .* f.(A) ./ A
      @. A + 2 * f(A) / A

[30]: 3×3 Matrix{Float64}:
        3.0   6.0   9.0
       12.0  15.0  18.0
       21.0  24.0  27.0

[31]: broadcast(x -> x + 2 * f(x) / x, A)

[31]: 3×3 Matrix{Float64}:
        3.0   6.0   9.0
       12.0  15.0  18.0
       21.0  24.0  27.0
```

Рис. 2.20: Варианты записи сложных выражений

2.4 Сторонние библиотеки (пакеты) в Julia

Julia имеет более 2000 зарегистрированных пакетов, что делает их огромной частью экосистемы Julia. Есть вызовы функций первого класса для других языков, обеспечивающие интерфейсы сторонних функций. Можно вызвать функции из Python или R, например, с помощью PyCall или Rcall.

При первом использовании пакета в текущей установке Julia необходимо использовать менеджер пакетов, чтобы явно его добавить:

```
[32]: import Pkg
      Pkg.add("Example")
```

```
    Updating registry at `C:\Users\eldar\.julia\registries\General.toml`
   Resolving package versions...
   Installed Example — v0.5.5
    Updating `C:\Users\eldar\.julia\environments\v1.12\Project.toml`
  [7876af07] + Example v0.5.5
    Updating `C:\Users\eldar\.julia\environments\v1.12\Manifest.toml`
  [7876af07] + Example v0.5.5
Precompiling packages...
   1168.0 ms  ✓ Example
  1 dependency successfully precompiled in 3 seconds. 49 already precompiled.
```

Рис. 2.21: Добавление стороннего пакета

При каждом новом использовании Julia (например, в начале нового сеанса в REPL или открытии блокнота в первый раз) нужно загрузить пакет, используя ключевое слово using:

- Например, добавим и загрузим пакет Colors. Затем создадим палитру из 100 разных цветов, а затем определим матрицу 3×3 с элементами в форме случайного цвета из палитры, используя функцию rand:

```
[33]: Pkg.add("Colors")
using Colors
palette = distinguishable_colors(100)
rand(palette, 3, 3)
```

```
   Resolving package versions...
   Installed Statistics ──────── v1.11.1
   Installed Reexport ─────────── v1.2.2
   Installed FixedPointNumbers ─ v0.8.5
   Installed ColorTypes ──────── v0.12.1
   Installed Colors ───────────── v0.13.1
    Updating `C:\Users\eldar\.julia\environments\v1.12\Project.toml`
  [5ae59095] + Colors v0.13.1
    Updating `C:\Users\eldar\.julia\environments\v1.12\Manifest.toml`
  [3da002f7] + ColorTypes v0.12.1
  [5ae59095] + Colors v0.13.1
  [53c48c17] + FixedPointNumbers v0.8.5
  [189a3867] + Reexport v1.2.2
  [10745b16] + Statistics v1.11.1
Precompiling packages...
    517.9 ms  ✓ Reexport
    703.9 ms  ✓ Statistics
   2016.6 ms  ✓ FixedPointNumbers
   1519.1 ms  ✓ ColorTypes
    668.4 ms  ✓ ColorTypes → StyledStringsExt
   4498.7 ms  ✓ Colors
  6 dependencies successfully precompiled in 9 seconds. 50 already precompiled.
```

[33]:

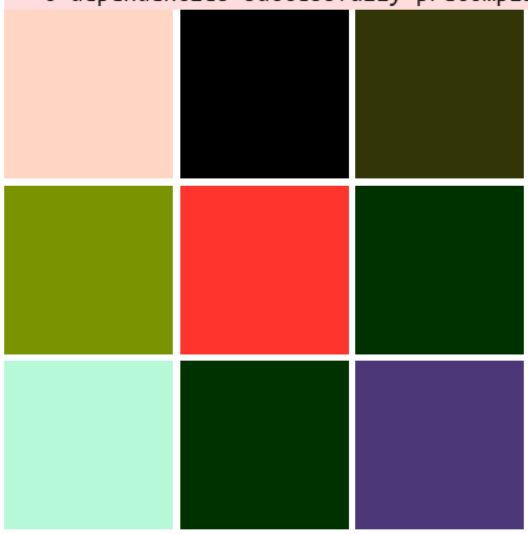

Рис. 2.22: Создание матрицы цветов

2.5 Задания для самостоятельного выполнения

1)

```
[35]: for i in 1:100
          println("$i -> $(i^2)")
      end
```

```
1 -> 1
2 -> 4
3 -> 9
4 -> 16
5 -> 25
6 -> 36
7 -> 49
8 -> 64
9 -> 81
10 -> 100
```

Рис. 2.23: Задание №1.1: цикл for

```
[36]: i = 1
      while i <= 100
          println("$i -> $(i^2)")
          i += 1
      end
```

```
1 -> 1
2 -> 4
3 -> 9
4 -> 16
5 -> 25
6 -> 36
7 -> 49
8 -> 64
9 -> 81
10 -> 100
```

Рис. 2.24: Задание №1.1: цикл while

```
[37]: squares = Dict{Int, Int}()
for i in 1:100
    squares[i] = i^2
end

println("Первые 5 элементов словаря:")
for (k, v) in pairs(squares)
    if k <= 5
        println("$k -> $v")
    end
end
```

```
Первые 5 элементов словаря:
5 -> 25
4 -> 16
2 -> 4
1 -> 1
3 -> 9
```

Рис. 2.25: Задание №1.2: цикл for

```
[38]: squares = Dict{Int, Int}()
i = 1
while i <= 100
    squares[i] = i^2
    i += 1
end
println("Последние 5 элементов словаря:")
for (k, v) in pairs(squares)
    if k <= 5
        println("$k -> $v")
    end
end
```

```
Последние 5 элементов словаря:
5 -> 25
4 -> 16
2 -> 4
1 -> 1
3 -> 9
```

Рис. 2.26: Задание №1.2: цикл while

```
[40]: squares_arr = Int[]
for i in 1:100
    push!(squares_arr, i^2)
end
println("Первые 5 квадратов: ", squares_arr[1:5])
println("Последние 5 квадратов: ", squares_arr[96:100])

Первые 5 квадратов: [1, 4, 9, 16, 25]
Последние 5 квадратов: [9216, 9409, 9604, 9801, 10000]
```

Рис. 2.27: Задание №1.3: цикл for

```
[41]: squares_arr = Int[]
i = 1
while i <= 100
    push!(squares_arr, i^2)
    i += 1
end
println("Первые 5 квадратов: ", squares_arr[1:5])
println("Последние 5 квадратов: ", squares_arr[96:100])

Первые 5 квадратов: [1, 4, 9, 16, 25]
Последние 5 квадратов: [9216, 9409, 9604, 9801, 10000]
```

Рис. 2.28: Задание №1.3: цикл while

2)

```
[45]: println("Введите число: ")
n = parse(Int, readline())
if(n % 2 == 0)
    println(n)
else
    println("нечётное")
end

println(n%2==0 ? n : "нечётное")

Введите число:
stdin>  3
нечётное
нечётное
```

Рис. 2.29: Задание №2: результат 1

```
[46]: println("Введите число: ")
      n = parse(Int, readline())
      if(n % 2 == 0)
          println(n)
      else
          println("нечётное")
      end

      println(n%2==0 ? n : "нечётное")

      Введите число:
      stdin>  2
      2
      2
```

Рис. 2.30: Задание №2: результат 2

3)

```
[47]: add_one(x) = x+1

[47]: add_one (generic function with 1 method)

[48]: add_one(4)

[48]: 5
```

Рис. 2.31: Задание №3

4)

```
[50]: n, m = 2, 3
      linear_data = 1:(n * m)
      A = reshape(linear_data, n, m) .+ 0
      A

[50]: 2×3 Matrix{Int64}:
       1  3  5
       2  4  6
```

Рис. 2.32: Задание №4

5)

```
[53]: A = [1 1 3; 5 2 6; -2 -1 -3]
      A_cubed = A^3
      println("Матрица A:")
      println(A)
      println("\nA^3:")
      println(A_cubed)

      Матрица A:
      [1 1 3; 5 2 6; -2 -1 -3]

      A^3:
      [0 0 0; 0 0 0; 0 0 0]
```

Рис. 2.33: Задание №5.1

```
[56]: A = [1 1 3; 5 2 6; -2 -1 -3]
      println("Исходная матрица A:")
      println(A)
      A[:, 3] = A[:, 2] + A[:, 3]
      println("\nМатрица A после замены третьего столбца:")
      A

      Исходная матрица A:
      [1 1 3; 5 2 6; -2 -1 -3]

      Матрица A после замены третьего столбца:

[56]: 3×3 Matrix{Int64}:
        1   1   4
        5   2   8
       -2  -1  -4
```

Рис. 2.34: Задание №5.2

6)

```
[57]: B = zeros(15, 3)
      B[:, 1] .= 10
      B[:, 2] .= -10
      B[:, 3] .= 10
      println("Матрица B:")
      println(B)
      C = B' * B
      println("\nМатрица C = B^T * B:")
      println(C)

      Матрица B:
      [10.0 -10.0 10.0; 10.0 -10.0 10.0; 10.0 -10.0 10.0; 10.0 -10.0 10.0; 10.0 -10.0 10.0; 10.0 -10.0 10.0; 10.0 -10.0 10.0; 10.0 -10.0 10.0; 10.0 -10.0 10.0; 10.0 -10.0 10.0; 10.0 -10.0 10.0; 10.0
      -10.0 10.0; 10.0 -10.0 10.0; 10.0 -10.0 10.0; 10.0 -10.0 10.0; 10.0 -10.0 10.0]

      Матрица C = B^T * B:
      [1500.0 -1500.0 1500.0; -1500.0 1500.0 -1500.0; 1500.0 -1500.0 1500.0]
```

Рис. 2.35: Задание №6

7)

```
Z = zeros(Int, 6, 6)
E = ones(Int, 6, 6)

Z1 = copy(Z)
for i in 1:5
    Z1[i, i+1] = 1
    Z1[i+1, i] = 1
end

Z2 = copy(Z)
for i in 1:6
    for j in 1:6
        if (i + j) % 2 == 1
            Z2[i, j] = 1
        end
    end
end

Z3 = copy(Z)
for i in 1:6
    for j in 1:6
        if abs(i - j) == 2
            Z3[i, j] = 1
        end
    end
end
```

Рис. 2.36: Задание №7.1

```
Z3[1,4] = 1; Z3[1,6] = 1
Z3[2,3] = 1; Z3[2,5] = 1
Z3[3,2] = 1; Z3[3,6] = 1
Z3[4,1] = 1; Z3[4,5] = 1
Z3[5,2] = 1
Z3[6,1] = 1; Z3[6,3] = 1

Z4 = copy(Z)
for i in 1:6
    for j in 1:6
        if (i + j) % 2 == 0
            Z4[i, j] = 1
        end
    end
end

println("\nМатрица Z₁:")
println(Z1)
println("\nМатрица Z₂:")
println(Z2)
println("\nМатрица Z₃:")
println(Z3)
println("\nМатрица Z₄:")
println(Z4)
```

Рис. 2.37: Задание №7.2

```
Матрица Z₁:
[0 1 0 0 0 0; 1 0 1 0 0 0; 0 1 0 1 0 0; 0 0 1 0 1 0; 0 0 0 1 0 1; 0 0 0 0 1 0]

Матрица Z₂:
[0 1 0 1 0 1; 1 0 1 0 1 0; 0 1 0 1 0 1; 1 0 1 0 1 0; 0 1 0 1 0 1; 1 0 1 0 1 0]

Матрица Z₃:
[0 0 1 1 0 1; 0 0 1 1 1 0; 1 1 0 0 1 1; 1 1 0 0 1 1; 0 1 1 0 0 0; 1 0 1 1 0 0]

Матрица Z₄:
[1 0 1 0 1 0; 0 1 0 1 0 1; 1 0 1 0 1 0; 0 1 0 1 0 1; 1 0 1 0 1 0; 0 1 0 1 0 1]
```

Рис. 2.38: Задание №7.3

8)

```
[16]: function outer(x, y, operation)
          return operation.(x[:,], y')
      end
      i = 0:4
      j = 0:4
      A1 = outer(i, j, (x,y) -> x + y)
      x = 0:4
      y = [1, 2, 4, 8, 16]
      A2 = outer(x, y, (a,b) -> b^a)
      i = 0:4
      j = 0:4
      A3 = outer(i, j, (x,y) -> mod1(x + y, 5) - 1)
      n = 10
      i = 0:n-1
      j = 0:n-1
      A4 = outer(i, j, (x,y) -> mod1(x + y, n) .- 1)
      i = 0:8
      j = 0:8
      A5 = outer(i, j, (x,y) -> 8 - abs(x - y))
      println("A1:\n", A1)
      println("A2:\n", A2)
      println("A3:\n", A3)
      println("A4:\n", A4)
      println("A5:\n", A5)

      A1:
      [0 1 2 3 4; 1 2 3 4 5; 2 3 4 5 6; 3 4 5 6 7; 4 5 6 7 8]
      A2:
      [1 1 1 1 1; 1 2 4 8 16; 1 4 16 64 256; 1 8 64 512 4096; 1 16 256 4096 65536]
      A3:
      [4 0 1 2 3; 0 1 2 3 4; 1 2 3 4 0; 2 3 4 0 1; 3 4 0 1 2]
      A4:
      [9 0 1 2 3 4 5 6 7 8; 0 1 2 3 4 5 6 7 8 9; 1 2 3 4 5 6 7 8 9 0; 2 3 4 5 6 7 8 9 0 1; 3 4 5 6 7 8 9 0 1 2; 4 5 6 7 8 9 0 1 2 3; 5 6 7 8 9 0 1 2 3 4; 6 7 8 9 0 1 2 3 4 5; 7 8 9
      0 1 2 3 4 5 6; 8 9 0 1 2 3 4 5 6 7]
      A5:
      [8 7 6 5 4 3 2 1 0; 7 8 7 6 5 4 3 2 1; 6 7 8 7 6 5 4 3 2; 5 6 7 8 7 6 5 4 3; 4 5 6 7 8 7 6 5 4; 3 4 5 6 7 8 7 6 5; 2 3 4 5 6 7 8 7 6; 1 2 3 4 5 6 7 8 7; 0 1 2 3 4 5 6 7 8]
```

Рис. 2.39: Задание №8

9)

```
[18]: using LinearAlgebra

      A = [1  2  3  4  5;
           2  1  2  3  4;
           3  2  1  2  3;
           4  3  2  1  2;
           5  4  3  2  1]

      b = [7; -1; -3; 5; 17]
      lu_result = lu(A)
      x = A \ b
      x

[18]: 5-element Vector{Float64}:
       -2.0000000000000036
        3.0000000000000058
        4.999999999999998
        1.9999999999999991
       -3.999999999999999
```

Рис. 2.40: Задание №9

10)

```
[19]: using Random
Random.seed!(123)
M = rand(1:10, 6, 10)
println("Матрица M:")
println(M)
N = 4
count_greater_than_N = sum(M .> N, dims=2)
println("\nКоличество элементов в каждой строке, больших $N:")
println(vec(count_greater_than_N))

M_value = 7
occurrences = sum(M .== M_value, dims=2)
rows_with_two_occurrences = findall(occurrences .== 2)
println("\nСтроки, где число $M_value встречается ровно 2 раза:")
println(rows_with_two_occurrences)

K = 75
column_pairs = Tuple{Int, Int}[]
for j1 in 1:size(M, 2)
    for j2 in (j1 + 1):size(M, 2)
        if sum(M[:, j1] + M[:, j2]) > K
            push!(column_pairs, (j1, j2))
        end
    end
end

println("\nПары столбцов, сумма элементов которых больше $K:")
println(column_pairs)
```

```
Матрица M:
[6 1 1 7 4 3 4 4 2 7; 6 10 5 3 2 1 1 9 6 10; 9 6 4 3 1 6 1 3 9 8; 2 4 2 10 4 4 10 10 3 7; 6 6 8 9 5 2 4 9 9 9; 4 9 6 3 8 3 8 8 4 5]

Количество элементов в каждой строке, больших 4:
[3, 6, 5, 4, 8, 6]

Строки, где число 7 встречается ровно 2 раза:
CartesianIndex{2}[CartesianIndex(1, 1)]

Пары столбцов, сумма элементов которых больше 75:
[(1, 8), (1, 10), (2, 8), (2, 10), (4, 8), (4, 10), (8, 9), (8, 10), (9, 10)]
```

Рис. 2.41: Задание №10

11)

```
[21]: sum1 = 0.0
sum2 = 0.0
# Первая сумма: sum(i=1 to 20) sum(j=1 to 5) i^4 / (3 + i*j)
for i in 1:20
    for j in 1:5
        sum1 += i^4 / (3 + j)
    end
end
# Вторая сумма: sum(i=1 to 20) sum(j=1 to 5) i^4 / (3 + j)
for i in 1:20
    for j in 1:5
        sum2 += i^4 / (3 + i * j)
    end
end

println("Первая сумма: ", sum1)
println("Вторая сумма: ", sum2)
```

```
Первая сумма: 639215.2833333334
Вторая сумма: 89912.02146097136
```

Рис. 2.42: Задание №11

3 Выводы

В результате выполнения лабораторной работы, я освоил применение циклов функций и сторонних для Julia пакетов для решения задач линейной алгебры и работы с матрицами.